AFIAKOS — Performance Overview & Bottleneck Analysis

Measured via `tb_crypto_bench.vhd` (GHDL simulation, `--std=08`).
Configuration: **100 MHz clock**, **4-cycle AXI-Lite read latency** (behavioural DDR model).
Run with: `make bench` from `vhdl/crypto/`.

Test environment

Parameter	Value
Clock	100 MHz
Memory model read latency	4 cycles (AR → R valid)
Memory model write latency	1 cycle (AW+W simultaneous → B next cycle)
DMA word width	32-bit AXI-Lite (4 transactions per 128-bit block)
Key size	256 bits (CAST-256)
IV size	96 bits
GCM tag	128 bits

S1 — AXI control-plane latency

Single register access measured end-to-end (AW/W accepted → B valid for writes; AR accepted → R valid for reads).

Operation	Cycles	Time @ 100 MHz
Register write (one word)	**3 cy**	30 ns
Register read (one word)	**4 cy**	40 ns

Operation	Cycles	Time @ 100 MHz
Full key load (8 writes)	31 cy	310 ns

Control-plane overhead is negligible compared to cryptographic latency.

S2 — Key-schedule + H-precompute overhead

The GCM state machine runs a full CAST-256 key schedule (KS) on every `START`, then computes the GHASH subkey H = BC(K, 0).
A **key cache** inside `gcm.vhd` detects when the same key is reused and skips both steps.

Mode	Cycles	Time @ 100 MHz	Breakdown
Cold (new key / after soft-reset)	**515 cy**	5.15 µs	KS ≈ 400 cy + H ≈ 53 cy + ARM states ≈ 62 cy
Warm (same key, cache hit)	**64 cy**	0.64 µs	GCTR_INIT = 2 cy + GHASH-final + tag = 57 cy + overhead = 5 cy

Cache saving per hit: 451 cy (4.51 µs).

The warm figure (64 cy) represents the minimum fixed overhead for any GCM operation regardless of payload size — it cannot be reduced without changing the GHASH-final or tag computation.

S3 — Encrypt throughput (warm key, AAD = 0)

Each row is one `START → IRQ` cycle. Columns:

Column	Meaning
`total`	Cycles from START-ack to IRQ
`ks+h`	Cycles before first DMA read (key-schedule + H; 0 when warm + no fixed setup)
`rd`	Cycles from first AR-valid to last R-valid (DMA read window)

Column	Meaning
`wr`	Cycles from first AW-valid to last B-valid (DMA write window)
`post`	Cycles from last AXI activity to IRQ (GHASH-final + tag)
`Mbps`	Payload throughput at 100 MHz

PT blocks	Payload	total	ks+h	rd	wr	post	Mbps
1	16 B	137	13	21	13	57	**93**
2	32 B	189	13	45	65	57	**135**
3	48 B	241	13	69	117	57	**159**
4	64 B	293	13	93	169	57	**174**
5	80 B	345	13	117	221	57	**185**
6	96 B	397	13	141	273	57	**193**
7	112 B	449	13	165	325	57	**199**
8	128 B	501	13	189	377	57	**204**

Throughput is converging toward an asymptote — 8 blocks reaches **83% of it** (see analysis below).

Bottleneck analysis

Per-block marginal cost: 52 cy

Derived from S3 data:

```
marginal = (total[N=8] - total[N=1]) / 7 = (501 - 137) / 7 = 52 cy/block
```

Breaking the 52 cy/block down into its three sequential phases:

Phase	Cycles / block	Derivation
DMA read (4 AXI-Lite reads per 128-bit block)	**24 cy**	`rd` column grows by 24 per block
CAST-256 encrypt (cipher latency, key cached)	**~15 cy**	residual after subtracting DMA read and write
DMA write (4 AXI-Lite writes per 128-bit block)	**~13 cy**	`wr[N=1]` = 13 cy (first block baseline)

> **Why the `wr` column grows faster than `rd`:**
> The write window is measured from the *first* AW (block 0 write start) to the *last* B (block N write complete). Between these two events, the pipeline also runs CAST-256 on subsequent blocks, stretching the window. The 52 cy/block write-span growth telescopes the full interleaved sequence: read → encrypt → write → read → encrypt → write → …
> The read window (24 cy/block) is the true raw read bandwidth.

Fixed overhead per warm operation: 70 cy

```
fixed overhead = ks+h (13 cy) + post (57 cy)
```

- **13 cy** — GCTR counter initialisation and adapter setup after the key-cache hit
- **57 cy** — GHASH length-block processing + tag generation (constant regardless of payload)

This 70-cycle floor means operations on very short messages (1–2 blocks) are fixed-overhead dominated. The 1-block case spends **51% of its cycles** on fixed overhead.

Asymptotic throughput

```
bits per cycle  = 128 / 52 = 2.46 b/cy
throughput      = 2.46 × 100 MHz = 246 Mbps  (theoretical maximum, warm key, no AAD)
```

At 8 blocks the measured throughput is **204 Mbps** (83% of asymptote).
At 16 blocks it would reach approximately **225 Mbps** (91%).

Where time goes: visual breakdown (8-block warm encrypt, 501 cy total)

```
|-- ks+h (13) --|-- DMA + cipher (431 cy) --|-- GHASH-final (57) --|
                |<-- rd window: 189 cy ------->|
                        |<-- wr window: 377 cy ---------------------->|
```

The DMA read and write windows **overlap** — while encrypting block N and writing its CT, the DMA is already reading block N+1. The post-cipher GHASH-final runs after the last write.

Improvement opportunities

1. AXI-Lite → AXI4 burst DMA (highest impact)

Problem: Each 128-bit block requires 4 independent AXI-Lite transactions (24 cy read, ~13 cy write per block). AXI4 burst mode delivers all 4 words in a single transaction.

Expected gain: DMA read time drops from 24 cy/block to ~6–8 cy/block; write similarly. **Asymptotic throughput** would increase from ~246 Mbps to **~500–600 Mbps**.

This is the single most impactful change possible without touching the cipher.

2. CAST-256 block-level pipelining (medium impact)

Problem: CAST-256 encryption is sequential; block N must complete before block N+1 can start. The ~15 cy cipher stall cannot be hidden by the current DMA adapter.

Expected gain: Overlapping cipher and DMA (start encrypting block N+1 while writing block N CT) would reduce the marginal cost from 52 cy to approximately `max(DMA_read, cipher)` ≈ 24 cy/block. **Asymptotic throughput** would increase from ~246 Mbps to **~533 Mbps**.

Requires a 2-deep block pipeline in the GCM FSM and a 2-slot DMA read buffer.

3. Key-schedule caching (already implemented)

The warm/cold gap (451 cy) is already handled by the key cache in `gcm.vhd`. For applications that repeatedly encrypt with the same key (e.g., a session cipher) this is fully amortised.

Recommendation: Callers should write a new key only when the key actually changes, not on every operation setup. At 100 MHz, an unnecessary cold start costs 4.51 µs — equivalent to the encryption time of ~8 blocks.

4. GHASH-final latency (low impact)

The 57-cycle GHASH-final (LEN_HASH → DONE) is inherent to GCM: one CAST-256 block encrypt for the tag, one GHASH multiply for the length block. This cannot be reduced without algorithmic changes. It is most visible for 1-block messages (57/137 = 42% of total).

Summary table

Metric	Value
AXI write latency	3 cy (30 ns)
AXI read latency	4 cy (40 ns)
Cold start overhead (KS + H)	515 cy (5.15 µs)
Warm start overhead (fixed per-op)	64 cy (0.64 µs)
Marginal cost per 128-bit block (warm)	52 cy (520 ns)
DMA read bandwidth	24 cy / 128-bit block
DMA write bandwidth	~13 cy / 128-bit block
GHASH-final + tag	57 cy (constant)
Peak throughput (warm, large payload)	~246 Mbps (asymptote)
Measured throughput at 8 blocks	204 Mbps
Projected throughput with AXI4 bursts	~500–600 Mbps

How to reproduce

```
# From vhdl/crypto/
make bench            # default: 100 MHz, 4-cycle mem latency
make bench CLK_MHZ=200 MEM_LAT=2   # different clock / memory model
make bench-vcd        # same + dump waveform to tb_crypto_bench.vcd
```

The benchmark testbench is `vhdl/crypto/tb_crypto_bench.vhd`.

The integration correctness testbench is `vhdl/crypto/tb_crypto_integration.vhd` (`make run`).